

ENSEA

Beyond Engineering

REAL-TIME OPERATING SYSTEMS

Session 5 – Memory and Statistics

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today

1. **Lecture**: memory management in FreeRTOS
   - Where do our variables, stacks, tasks and queues live?
   - Dynamic or static allocation
2. **Guided lab**: statistics
   - Monitor the heap, the stacks, the CPU time
   - In the debugger (STM32CubeIDE) and in the console

ENSEA
Beyond Engineering

Today

1. Memory in a Bare-Metal Program
2. Memory in FreeRTOS
3. Dynamic or Static Allocation
4. Lab: Setup
5. Lab: the Heap
6. Lab: the Stacks
7. Lab: the CPU Time
8. Lab: the Queues
9. Conclusion

ENSEA
Beyond Engineering

Three ways to get memory

Static

```
uint8_t global = 12;
static uint8_t s = 142;
void f(void) {
    static uint8_t n = 0;
}
```

- Reserved by the linker
- Lives forever
- Checked at **link time**

Stack

```
void f(void) {
    uint8_t local = 0;
}
```

- Local variables
- Freed when the function returns
- No fragmentation, but can **overflow**

Heap

```
p = malloc(64);
q = xQueueCreate(16, 8);
```

- On demand, at run time
- Can **fail**, can fragment
- Hard to test

  Peux détruire une partie de l'espace allouer et donc des données

- Embedded systems: limited memory, timing constraints. Some coding standards forbid dynamic allocation, or allow it only before the scheduler starts

ENSEA
Beyond Engineering

Stack and heap in CubeMX

Screenshot
Project Manager > Linker Settings:
Minimum Heap Size, Minimum Stack Size

- **Stack**: the system stack (MSP)
  - Used before `main`, by `main`...
  - ...then only by the interrupts, once the scheduler runs (session 1)
- **Heap**: the C heap, for `malloc`
  - Our own `mallocs`...
  - ...and hidden ones: `printf` allocates its buffer

ENSEA
Beyond Engineering

Where are they defined?

Linker script (STM32G431xx_FLASH.ld)

```
_estack = ORIGIN(RAM) + LENGTH(RAM);      /* end of RAM */
_Min_Heap_Size = 0x200;                   /* required amount of heap  */
_Min_Stack_Size = 0x400;                  /* required amount of stack */

/* User_heap_stack section, used to check that there is enough RAM left */
._user_heap_stack (NOLOAD) :
{
    PROVIDE ( _end = . );
    . = . + _Min_Heap_Size;
    . = . + _Min_Stack_Size;
} >RAM
```

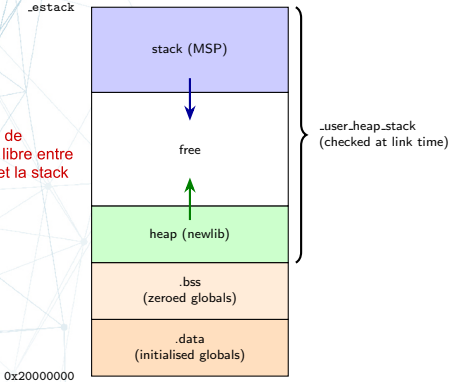
Startup (startup_stm32g431xx.s)

```
Reset_Handler:
  ldr   r0, =_estack
  mov   sp, r0          /* set stack pointer */
```

- The stack starts at the **top** of the RAM and grows **down**

ENSEA
Beyond Engineering

The bare-metal memory map



- The heap grows up, the stack grows down
- `_Min_Heap_Size` + `_Min_Stack_Size`: a **reservation**, checked by the linker
  - Counted in the "RAM used" figure
  - But not a limit: nothing stops the stack from going further at run time (no MPU)
- In bare metal, the stack can only collide with the heap – then with `.bss` and `.data`

ENSEA
Beyond Engineering

Who manages the C heap?

newlib (the C library) implements `malloc`. It gets memory through `_sbrk()`, in `Core/Src/sysmem.c`:

```
void *_sbrk(ptrdiff_t incr) {
    const uint32_t stack_limit = (uint32_t)&_estack - (uint32_t)&_Min_Stack_Size;
    const uint8_t *max_heap = (uint8_t *)stack_limit;

    if (NULL == __sbrk_heap_end) {
        __sbrk_heap_end = &_end;                    /* first call: just after .bss */
    }
    /* Protect heap from growing into the reserved MSP stack */
    if (__sbrk_heap_end + incr > max_heap) {
        errno = ENOMEM;
        return (void *)-1;
    }
    prev_heap_end = __sbrk_heap_end;
    __sbrk_heap_end += incr;
    return (void *)prev_heap_end;
}
```

- The heap may grow up to the stack reservation – the actual stack pointer is never checked

Stacks and heaps, in the plural

- Two stack pointers (session 1)
  - **MSP**: before the scheduler, then interrupts and kernel
  - **PSP**: tasks, each one on **its own stack**
- Two heaps
  - **C heap** (newlib): `malloc`, `printf`...
  - **FreeRTOS heap**: every `x...Create` – tasks (TCB + stack), queues, semaphores, timers...

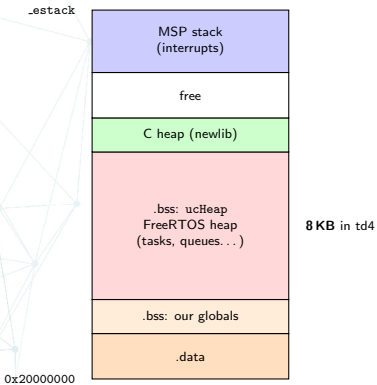
The FreeRTOS heap is... a static array

`static uint8_t ucHeap[configTOTAL_HEAP_SIZE];` in `heap_4.c`

The compiler sees one big global variable, in `.bss`. FreeRTOS manages what is inside.

ENSEA
Beyond Engineering

The FreeRTOS memory map



- The task stacks are *inside* the FreeRTOS heap
- The C heap can be reduced: only `printf` uses it here
- A bigger FreeRTOS heap = more room for `x...Create...`
- ...and less room for globals, buffers, the C heap and the MSP stack

ENSEA
Beyond Engineering

configTOTAL_HEAP_SIZE

Screenshot
CubeMX > FreeRTOS > Config parameters > TOTAL_HEAP_SIZE

- Generated in `Core/Inc/FreeRTOSConfig.h`

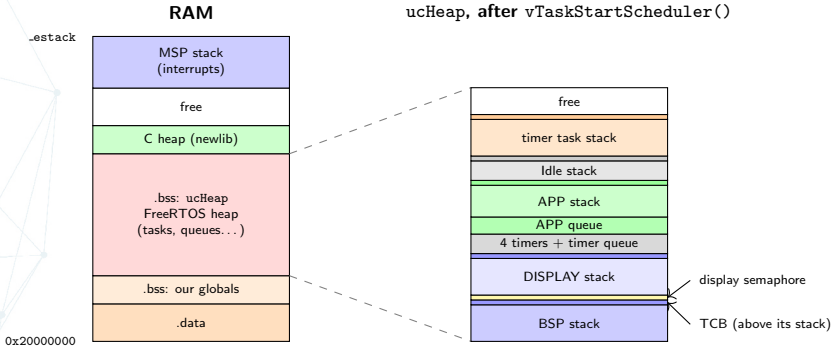
Did you notice?

	Heap	New objects
td1	3072	2 tasks + Idle
td2	4096	+ APP queue
td3	6144	+ display task, semaphore
td4	8192	+ timer task, queue, 4 timers

- The heap grew at every session – silently

ENSEA
Beyond Engineering

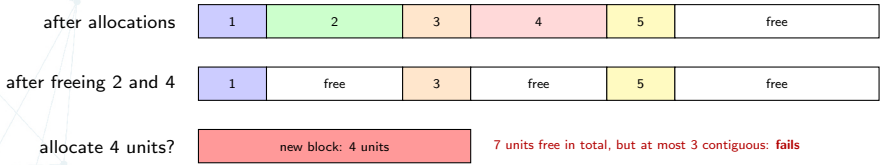
Inside the FreeRTOS heap (td4)



- Allocated in the order of the `x...Create` calls; the timer queue with the first `xTimerCreate`; Idle and timer task **in** `vTaskStartScheduler()`
- For each task: the stack first, then the TCB just above it (the stack grows down, away from its own TCB). Proportions are illustrative

ENSEA
Beyond Engineering

Fragmentation



- Enough free memory in total, but not enough **contiguous** memory
- Worse when memory is small – and invisible: free-heap figures do not show it

ENSEA
Beyond Engineering

Five heap implementations

File	Free?	Notes
`heap_1.c`	no	allocate only; simplest, deterministic
`heap_2.c`	yes	does not merge adjacent free blocks: same-size allocations only
`heap_3.c`	yes	wraps `malloc` / `free` of the C library, made thread-safe
`heap_4.c`	yes	merges adjacent free blocks; the default (CubeMX)
`heap_5.c`	yes	like heap_4, over several memory regions (internal + external RAM)

- Only one is compiled (CubeMX selects it: `MEMORY_ALLOCATION`)

And for us?

Everything is allocated at init, nothing is ever freed (no `vTaskDelete`, no `vQueueDelete`): **heap_1** would be enough.

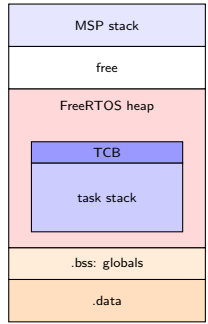
ENSEA
Beyond Engineering

Dynamic allocation

```
TaskHandle_t h_task;

if (xTaskCreate(task, "Task", STACK_DEPTH, NULL,
                PRIORITY, &h_task) != pdPASS) {
    Error_Handler();
}
```

- Can fail: not enough (contiguous) heap – check the return value
- Only the stack *size* is given
- `vTaskDelete` frees the memory (not with heap_1)

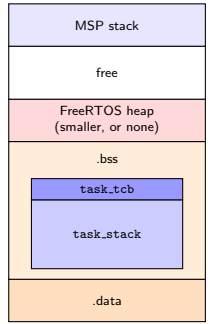


Static allocation

```
static StackType_t task_stack[STACK_DEPTH];
static StaticTask_t task_tcb;
TaskHandle_t h_task;

h_task = xTaskCreateStatic(task, "Task", STACK_DEPTH,
                           NULL, PRIORITY,
                           task_stack, &task_tcb);
```

- We provide the stack and the TCB: plain global variables
- Cannot fail for lack of memory: checked at **link time**
- `vTaskDelete` removes the task, but frees nothing



ENSEA
Beyond Engineering

Static allocation: the rest

- Every object has a static variant: `xQueueCreateStatic`, `xSemaphoreCreateBinaryStatic`, `xTimerCreateStatic`... – see the FreeRTOS API
- With `configSUPPORT_STATIC_ALLOCATION 1`, *you* must also provide the memory of the tasks created by the kernel:

```
void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCBBuffer,
                                   StackType_t **ppxIdleTaskStackBuffer,
                                   uint32_t *pulIdleTaskStackSize) {
    static StaticTask_t idle_tcb;
    static StackType_t idle_stack[configMINIMAL_STACK_SIZE];

    *ppxIdleTaskTCBBuffer = &idle_tcb;
    *ppxIdleTaskStackBuffer = idle_stack;
    *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}
/* same idea: vApplicationGetTimerTaskMemory(), if the timers are enabled */
```

- Without them, the project does not link

ENSEA
Beyond Engineering

Everything static?

Why static

- No failure at run time: errors at link time
- No fragmentation
- Required by some safety-critical coding standards

Why dynamic

- Simpler to write: one call, no buffers to declare
- The memory can be given back (`vTaskDelete`)

A common compromise

Dynamic allocation **only at init**, never afterwards: failures show up at the first boot, no fragmentation can build up – and heap_1 is enough.

ENSEA
Beyond Engineering

Lab overview

1. Setup: CubeMX, and a `stats` object that prints every second
2. The heap
3. The stacks: observe, then break
4. The CPU time
5. The queues
6. If time allows: measure the choices of the previous sessions

- Start from the td5 project: td4 + the `stats` object

ENSEA
Beyond Engineering

CubeMX, once and for all

Screenshot
CubeMX > FreeRTOS > Config parameters

Enable:

- `USE_TRACE_FACILITY`
- `USE_STATS_FORMATTING_FUNCTIONS`
- `GENERATE_RUN_TIME_STATS`
- `CHECK_FOR_STACK_OVERFLOW`
- `USE_MALLOC_FAILED_HOOK`

Then generate, build, run.

ENSEA
Beyond Engineering

The stats object (given)

```c
#define STATS_PERIOD_MS 1000

static void stats_task(void *unused) {
    (void)unused;
    TickType_t uptime_s = 0;

    for (;;) {
        vTaskDelay(pdMS_TO_TICKS(STATS_PERIOD_MS));
        uptime_s++;
        printf("\r\n--- uptime: %lu s ---\r\n", (unsigned long)uptime_s);
        /* TODO: heap, stacks, CPU time */
    }
}

void stats_init(unsigned int task_priority) {      /* like bsp_init: the object creates its task */
    BaseType_t ok = xTaskCreate(stats_task, "STATS", STATS_STACK_DEPTH, NULL, task_priority, NULL);
    configASSERT(ok == pdPASS);
}
```

- Low priority. Why not simply print from an APP handler?
  - `printf` on the UART, in polling: tens of milliseconds – breaks run-to-completion
  - The statistics functions suspend the scheduler while they run: debug tools

ENSEA
Beyond Engineering

The heap

1. After the init, does the FreeRTOS heap still change in our projects? non car les allocations on lieux avant le lancement de scheduler
2. Print the heap size and its occupancy in the `stats` task
   - `configTOTAL_HEAP_SIZE`, `xPortGetFreeHeapSize()`, `xPortGetMinimumEverFreeHeapSize()`
3. What happens when an allocation fails?
   - Check the return value of every `x...Create`
   - Write `vApplicationMallocFailedHook()`: why have both?
   - Why do we now call `Error_Handler()` after `vTaskStartScheduler()`?
4. Look at the static RAM after the build: what changes when you change `TOTAL_HEAP_SIZE`?

- Note: no function tells you about fragmentation

ENSEA
Beyond Engineering

The heap: what to see

Screenshot
console: heap size, free, minimum ever free

Screenshot
Build Analyzer: RAM usage

- The heap does not change after `vTaskStartScheduler()`: Idle and the timer task are its last allocations
- The static RAM includes the whole FreeRTOS heap, used or not

ENSEA
Beyond Engineering

Observe: stack usage in the debugger

Screenshot
Window > Show View > FreeRTOS > FreeRTOS Task List, with Toggle Stack Checking

1. Run in debug mode, then pause
2. Open the FreeRTOS Task List
3. Enable stack checking
4. Which task uses the largest part of its stack?

ENSEA
Beyond Engineering

Observe: stack usage in the console

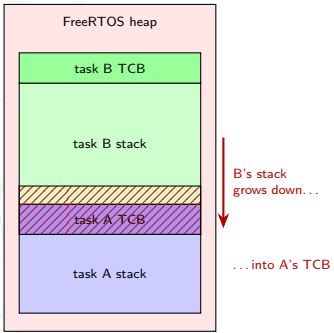
```
UBaseType_t free_words = uxTaskGetStackHighWaterMark(handle);   /* NULL: the calling task */

static char buffer[512];
vTaskList(buffer);              /* one line per task: name, state, priority, stack, number */
printf("%s", buffer);
```

- **High water mark**: the smallest amount of free stack ever seen, in words
- How does FreeRTOS know? At creation, every stack is filled with a known pattern (`0xA5`). It looks for how far the pattern was overwritten
  - Same technique in the debugger view, and in the overflow detection (method 2)
- `vTaskList` uses `sprintf`: watch the stack of the `stats` task itself

ENSEA
Beyond Engineering

Stack overflow



- Task B needs more stack than it was given
- Its stack grows down **below its base**: into the previous allocation – here, task A's TCB
- Nothing happens in task B. Task A crashes, later

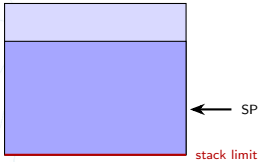
Why it is dangerous

It corrupts data and crashes **somewhere else**, at another time.

ENSEA
Beyond Engineering

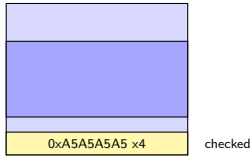
Detecting it: configCHECK_FOR_STACK_OVERFLOW

Method 1



- At each context switch: is the saved SP still inside the stack?
- Fast, but misses an overflow that came back before the switch

Method 2



- Method 1, plus: are the last 16 bytes of the stack still `0xA5`?
- Slower, catches more – but not a big local array that jumps over them

- A third method exists in the documentation, for other ports; CubeMX offers 1 and 2

ENSEA
Beyond Engineering

The hook

```
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {
    (void)xTask;
    (void)pcTaskName;                    /* which task overflowed */
    HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
    for (;;) { }                         /* put a breakpoint here */
}
```

- Called by FreeRTOS, during a context switch: the system is already damaged, do the minimum
- A LED and a breakpoint: then read `pcTaskName` in the debugger

And the MSP?

These checks only cover the **task** stacks (PSP). The interrupt stack (MSP) is watched by nobody: it only depends on `_Min_Stack_Size`.

ENSEA
Beyond Engineering

Break it

1. Pick the task whose stack is the fullest (previous step)
2. Reduce its stack size until it overflows
3. Check that the hook catches it: LED, breakpoint, task name
4. Try both methods

Screenshot
debugger stopped in vApplicationStackOverflowHook

ENSEA
Beyond Engineering

Run-time statistics: the principle

- At each context switch, FreeRTOS reads a time counter and adds the elapsed time to the task that was running
- You provide the counter (`GENERATE_RUN_TIME_STATS`):
  - `void configureTimerForRunTimeStats(void);` – starts the timer
  - `unsigned long getRunTimeCounterValue(void);` – returns its value, on 32 bits
- Recommended frequency: 10 to 100 times the tick
  - Here: **TIM7** at 10 kHz (prescaler), `ARR = 0xFFFF`

Problem

`unsigned long` is 32 bits; TIM7 is a 16-bit timer: it overflows every 6.5 s.

ENSEA
Beyond Engineering

A 32-bit counter from a 16-bit timer (given)

```c
void configureTimerForRunTimeStats(void) {
    HAL_TIM_Base_Start(&htim7);                    /* 10 kHz, ARR = 0xFFFF */
}

unsigned long getRunTimeCounterValue(void) {
    static uint16_t prev = 0;
    static uint32_t total = 0;

    uint16_t cnt = __HAL_TIM_GET_COUNTER(&htim7);
    total += (uint16_t)(cnt - prev);               /* 16-bit difference: correct across an overflow */
    prev = cnt;
    return total;
}
```

- No interrupt needed, but one condition: the function must be called at least once per overflow (6.5 s)
  - It is called at every context switch, and the BSP wakes up every 5 ms: fine
- 32 bits at 10 kHz: overflows after about 5 days

ENSEA
Beyond Engineering

CPU time: what to see

Screenshot
FreeRTOS Task List: Run Time column

1. Show the run time of each task in the debugger
2. Print `vTaskGetRunTimeStats(buffer)` in the `stats` task
3. Which task uses the CPU the most? What percentage goes to Idle?

ENSEA
Beyond Engineering

Queues and semaphores in the debugger

Screenshot
FreeRTOS Queue List

- The debugger lists the queues and semaphores...
- ...but without names. Give them one:

```
vQueueAddToRegistry(app.events,
                    "APP events");
vQueueAddToRegistry(
    ctx->display_request,
    "display request");
```

- How full does the APP queue get?

ENSEA
Beyond Engineering

If time allows: measure the whole course

- Add the run-time statistics to the previous projects, and measure the Idle time

Project	What changed	Idle
td1	two tasks, the APP in a busy loop	... %
td2	APP active object, polling every 10 ms	... %
td3	display: DMA + dedicated task	... %
td4	software timers, `portMAX_DELAY`	... %

The numbers behind the course

Every session freed some CPU time. Here is the proof.

ENSEA
Beyond Engineering

Summary

- Static memory is checked at link time; the heap can fail and fragment; stacks can overflow
- Two stacks pointers (MSP, PSP), two heaps (newlib, FreeRTOS)
- The FreeRTOS heap is one big static array in `.bss`: task stacks and TCBs live in it
- Five heap implementations; with allocations at init only, heap_1 is enough
- Static allocation: no run-time failure, but more code
- Monitor: free heap, stack high water marks, overflow hook, run-time statistics
- A stack overflow crashes *somewhere else*: detect it, do not guess it

La variable est dans la pile qui est dans l'etat

ENSEA
Beyond Engineering